

LC 3244: Shortest Distance After Road Addition Queries II

Context & Observation

Comparing constraints between similar problems:

- In LC 3243: $n \leq 500, q \leq 500$
- In LC 3244: $n = 100,000, q = 100,000$

Running BFS after every query is **not feasible** for LC 3244.

Observation:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

If we have a query $2 \rightarrow 6$, the path becomes:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 6 \rightarrow 7$

Nodes **3, 4, and 5** will never appear again. So when we add an edge $u \rightarrow v$, the intermediate nodes $\{u+1, u+2, \dots, v-1\}$ become useless.

Hence, instead of a standard adjacency list, we can store and maintain a **next[node]** array.

Initially:

$\text{next}[0] = 1$

$\text{next}[1] = 2$

...

$\text{next}[n-2] = n-1$

Algorithm

- S1 Maintain a `next[]` array.
- S2 `answer = current shortest distance`
- S3 For every query, if (`next[u] < v`), keep walking:
`u → next[u] → next[next[u]] ...`
- S4 Until you reach `v`.
- S5 `answer--` (decrement for every skipped node).
- S6 Finally, update `next[u] = v`.

C++ Implementation

```
class Solution {
public:
    vector<int> shortestDistanceAfterQueries(int n, vector<vector<int>>&
queries) {

        vector<int> next(n);
        for(int i = 0; i < n; i++){
            next[i] = i + 1;
        }

        int ans = n - 1;
        vector<int> res;

        for(auto &it : queries){
            int u = it[0];
            int v = it[1];

            int curr = u;

            while(next[curr] < v){
                ans--;
                int nextnode = next[curr];
                next[curr] = v;
            }

            res.push_back(ans);
            curr = v;
        }

        return res;
    }
};
```

```
        curr = nextnode;
    }

    res.push_back(ans);
}
return res;
}
};
```